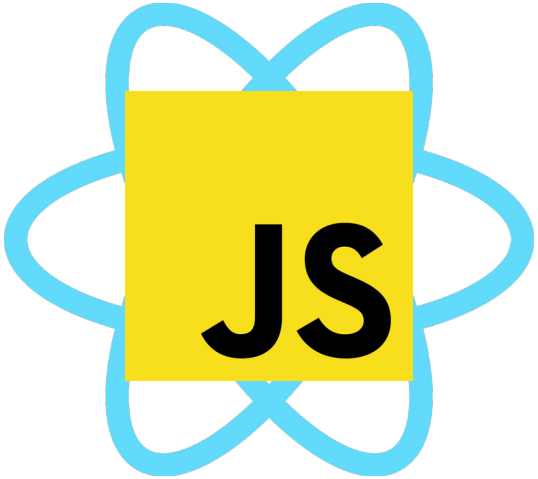


JavaScript in React

where JavaScript ends and React begins

Example: PostCard

```
function PostCard({ title, caption, likes }) {
 const [liked, setLiked] = useState(false);
 return (
  <div className="post-card"style={{ padding: 10, margin: 5 }} >
    <h3> { title } </h3>
     <p> { caption} </p>
     <p> { likes } likes </p>
     <button onClick={ () => setLiked(!liked) } >
        { liked ? "Unlike" : "Like" }
     </button>
  </div>
  );
}
export default PostCard;
```

Content

JavaScript in React code

- Which parts of React code are JavaScript and which are React-specific?
- Which JavaScript techniques are essential for writing React components?

What is purely React?

- What does React add that JavaScript alone does not have?

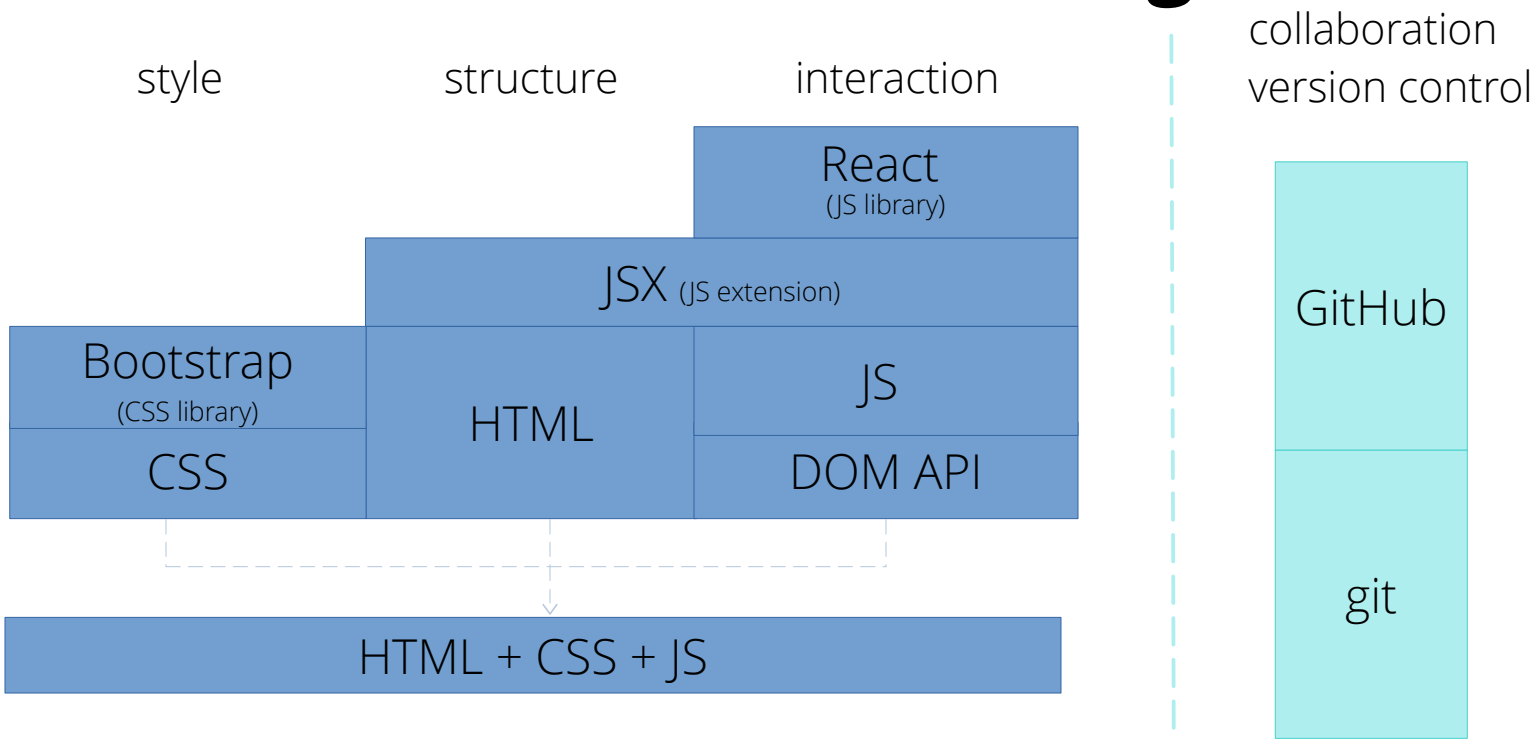
What is purely JavaScript?

- Which parts of React code work exactly the same outside React?
- What is the "border zone" — JavaScript that behaves differently inside React?

From JavaScript to React — same app, different approach

- How does the same feature look in plain JavaScript vs. React?
- What changed, and what stayed the same?

Reminder: Course technologies



JavaScript in React code

- Which parts of React code are JavaScript and which are React-specific?
- Which JavaScript techniques are essential for writing React components?

What is purely React?

- What does React add that JavaScript alone does not have?

What is purely JavaScript?

- Which parts of React code work exactly the same outside React?
- What is the "border zone" — JavaScript that behaves differently inside React?

From JavaScript to React — same app, different approach

- How does the same feature look in plain JavaScript vs. React?
- What changed, and what stayed the same?

How JSX becomes JavaScript

WE WRITE JSX (.jsx files)

↓

Vite serves the file

↓

Babel transpiles JSX to JavaScript

↓

BROWSER RUNS JavaScript

JSX transpiles to React.createElement()...

we write JSX

```
function Hello() {                §JS
  return                          §REACT
    <span>Hello!</span>           §HTML
  ;                               §JS
}                                 §JS
```

Babel outputs JavaScript

```
function Hello() {                §JS
  return                          §JS
    React.createElement(          §REACT
      'span',                     §JS
      null,                       §JS
      'Hello!'                    §JS
    );                            §JS
}                                 §JS
```

JSX is an extension to JS syntax, JS+XML.

React is a JS library.

JSX is a convenience for the developer, the actual runtime code is plain JavaScript that calls React's functions.

```
JavaScript
React
HTML/JSX
```

...so React code IS JavaScript

Greeting.jsx

```
function Greeting(props) {
  const message = "Welcome to React";
  return (
    <div>
      <h1>Hello,
      {
        props.name
      }
      !</h1>
      <p>
      {
        message
      }
      </p>
    </div>
  );
}
export default Greeting;
```

```
§JS
§JS
§REACT
§HTML
§HTML
§REACT
§JS
§REACT
§HTML
§HTML
§REACT
§JS
§REACT
§HTML
§HTML
§REACT
§JS
§REACT
```

React code IS JavaScript.
React is a JavaScript library.
React component is a JavaScript function.

JavaScript
React
HTML/JSX

...so React code IS JavaScript

Greeting.jsx

```
function Greeting (props) {
    const message = "Welcome to React";
  return (
      <div>
        <h1>Hello, { props.name }!</h1>
        <p>{ message }</p>
      </div>
  );
 }
export default Greeting;
```

(same code with more readable splitting to rows)

React code IS JavaScript.
React is a JavaScript library.
React component is a JavaScript function.

JavaScript
React
HTML/JSX

.map() — rendering lists

```
const cats = ["Toast", "Naranja", "Kinsky"];     §JS
const greetings = cats.map(cat =>                §JS
  "Hello, " + cat + "!"                          §JS
);                                               §JS
["Hello, Toast!", "Hello, Naranja!", "Hello, Kinsky!"]
```

JS basics — .map() returns new values from an array

```
const cats = ["Toast", "Naranja", "Kinsky"];     §JS
return (                                         §REACT
  cats.map((cat, i) =>                           §JS
    <Greeting key= {                             §REACT
        i                                        §JS
      }                                          §REACT
      name= {                                    §REACT
        cat                                      §JS
      }                                          §REACT
    />                                           §REACT
  )                                              §JS
)                                                §REACT
```

In React — .map() returns components from an array

JavaScript
React
HTML/JSX

Spread operator - immutable state updates

```
const cat = { name: "Toast", age: 6 };    §JS
const updated = { ...cat, age: 7 };       §JS
```

JS basics — spread copies an object

```
cat.isAsleep = true;                      §JS
setCat(cat);                              §REACT
```

Wrong way in React

```
setCat(                                   §REACT
  {                                       §JS
    ...cat,                               §JS
    isAsleep: true                        §JS
  }                                       §JS
);                                        §REACT
```

Right way in React

JavaScript
React
HTML/JSX

Ternary operator — conditional rendering

```
const greeting = isLoggedIn                §JS
  ? "Welcome!"                             §JS
  : "Please log in";                       §JS

return (                                   §REACT
  <div>                                    §HTML
    {                                      §REACT
      props.loggedIn                       §JS
      ? (                                  §JS
        <h1>Welcome back!</h1>             §HTML
      )                                    §JS
      : (                                  §JS
        <h1>Please log in.</h1>            §HTML
      )                                    §JS
    }                                      §REACT
  </div>                                   §HTML
);                                         §REACT
```

JS basics:
short if/else

In React:
if/else does not work inside JSX

JavaScript
React
HTML/JSX

Destructuring in JavaScript

```
const cat = {
  name: "Toast",
  age: 6,
  color: "orange"
};

// without destructuring
console.log(cat.name);   // "Toast"
console.log(cat.age);    // 6
console.log(cat.color);  // "orange"

// with destructuring
const { name, age, color } = cat;
console.log(name);   // "Toast"
console.log(age);    // 6
console.log(color);  // "orange"
```

Destructuring is a JavaScript feature that lets you unpack values from an object into variables.

JavaScript
React
HTML/JSX

Destructuring — handling props

```
function PostCard({ title, caption }) {  §JS
  return (                               §REACT
    <div>                                §HTML
      <h3>                               §HTML
      {                                  §REACT
        title                            §JS
      }                                  §REACT
      </h3>                              §HTML
      <p>                                §HTML
      {                                  §REACT
        caption                          §JS
      }                                  §REACT
      </p>                               §HTML
    </div>                               §HTML
  )                                      §REACT
}                                        §JS
```

```
function PostCard(props) {  §JS
  return                    §REACT
    <h3>                    §HTML
    {                       §REACT
      props.title           §JS
    }                       §REACT
    </h3>                   §HTML
}                           §JS
```

Destructuring works the same way inside and outside React. Without destructuring: receive the whole props object, write **props.title**, **props.caption** every time. With destructuring: unpack the props at the parameter list, write **title**, **caption** directly. Both versions produce the same result.

Arrow functions — components and callbacks

```
const twice = (x) => x * 2;                §JS
twice(5);                                  §JS

<button                                    §HTML
  onClick=                                 §REACT
  {                                        §REACT
    handleClick()                          §JS
  }                                        §REACT
>                                          §HTML
```

JS basics: reusable arrow function named twice that takes parameter x and returns its value multiplied by 2, using shorthand where the expression after => is automatically returned without needing an explicit return statement

```
<button                                    §HTML
  onClick=                                 §REACT
  {                                        §REACT
    () => handleClick(id)                  §JS
  }                                        §REACT
>                                          §HTML
```

→ WRONG (calls immediately when component renders)

```
<button                                    §HTML
  onClick=                                 §REACT
  {                                        §REACT
    handleClick                            §JS
  }                                        §REACT
>                                          §HTML
```

RIGHT (passes a function reference, called on click)

Also RIGHT for simple cases

JavaScript
React
HTML/JSX

Template literals — dynamic strings

```
const greeting = `Hello ${name}, welcome!`;     §JS
const url = `/api/products/${productId}`;       §JS
const imagePath = `/images/${filename}.png`;    §JS
```

```
<img src=                                       §HTML
  {                                             §REACT
    `/images/${filename}.png`                   §JS
  }                                             §REACT
/>                                              §HTML
<a href=                                        §HTML
  {                                             §REACT
    `/products/${productId}`                    §JS
  }                                             §REACT
>View product</a>                               §HTML
```

JS basics — template literals for dynamic strings.

React: the same syntax works inside JSX.

JavaScript
React
HTML/JSX

Template literals and ternary operator

```
const imagePath = `/images/${color}-${isAsleep ? 'sleeping' : 'awake'}.png`;  §JS
```

```
<img src=                                §HTML
  {                                      §REACT
    `/cats/${color}-${isAsleep           §JS
      ? 'sleeping'                       §JS
      : 'awake'}.png`                    §JS
  }                                      §REACT
  alt=                                   §HTML
  {                                      §REACT
    `${color} cat`                       §JS
  }                                      §REACT
/>                                       §HTML
```

React: the same syntax works inside JSX.

JavaScript
React
HTML/JSX

```
<img src= {  `/cats/${color}-${isAsleep ? 'sleeping' : 'awake'}.png`  } alt= {  `${color} cat`  } />
```

JavaScript in React code

- Which parts of React code are JavaScript and which are React-specific?
- Which JavaScript techniques are essential for writing React components?

What is purely React?

- What does React add that JavaScript alone does not have?

What is purely JavaScript?

- Which parts of React code work exactly the same outside React?
- What is the "border zone" — JavaScript that behaves differently inside React?

From JavaScript to React — same app, different approach

- How does the same feature look in plain JavaScript vs. React?
- What changed, and what stayed the same?

React hooks

JS does not have hooks

useState: state management for UI components

```
const [count, setCount] = useState(0); §REACT
```

useEffect: lifecycle and side effects

```
useEffect(() => {                          §REACT
  const timer = setTimeout(() => {         §JS
    console.log('done');                   §JS
  }, 1000);                                §JS
  return () => clearTimeout(timer);        §JS
}, []);                                    §REACT
```

JavaScript
React
HTML/JSX

JSX (JS XML) syntax

```
<div class="sidebar" onclick="handleClick()" >  §HTML
  <label for="name">Name</label>                §HTML
</div>                                          §HTML
```

HMTL: attribute named class and for

```
<div                                §HTML
  className="sidebar"               §REACT
  onClick= {                        §REACT
    handleClick                     §JS
  }                                 §REACT
>                                   §HTML
  <label                            §HTML
    htmlFor="name"                  §REACT
  >Name</label>                      §HTML
</div>                              §HTML
```

React:
- other names for reserved keywords
- { } for embedding JS

```
JavaScript
React
HTML/JSX
```

JSX tags — uppercase vs. lowercase

Lowercase tags = HTML elements (same as in regular HTML)

```
<div>                                  §HTML
<button>Click me</button>              §HTML
<img src="cat.png" />                  §HTML
<h1>Hello</h1>                         §HTML
<p>Some text</p>                       §HTML
```

- lowercase → HTML element (rendered as-is)

Uppercase tags = React components (created by you or imported from libraries)

```
<Greeting name="Toast" />              §REACT
<PostCard title="Hello" />             §REACT
<Card>                                 §REACT
<Button variant="primary">Click</Button>  §REACT
```

- Uppercase → React component (a function is called)

JavaScript
React
HTML/JSX

Components contain HTML tags (and other components)

```
function PostCard({ title }) {   §JS
  return (                       §REACT
    <div>                        §HTML
      <h3>                       §HTML
{                                §REACT
title                            §JS
}                                §REACT
</h3>                            §HTML
    </div>                       §HTML
  );                             §REACT
}                                §JS


<PostCard title="Hello" />       §REACT
```

The Uppercase <PostCard> is a component. The lowercase and <h3> are HTML tags that PostCard returns.

JavaScript
React
HTML/JSX

Props and one-way data binding

```
<PostList                                §REACT
  posts=                                 §REACT
  {                                      §REACT
    posts                                §JS
  }                                      §REACT
/>                                       §REACT

<PostForm                                §REACT
  onAddPost=                             §REACT
  {                                      §REACT
    addPost                              §JS
  }                                      §REACT
/>                                       §REACT


function handleSubmit() {                §JS
  props.onAddPost(newPost);              §REACT
}                                        §JS
```

JavaScript functions have arguments,
React adds data-down events-up convention.

Data flows DOWN via props

Events flow UP via callback props

Inside PostForm

JavaScript
React
HTML/JSX

JavaScript and DOM

```
let likes = 0;                                §JS
document.getElementById('likeButton')         §JS
  .addEventListener('click', () => {          §JS
    likes = likes + 1;                        §JS
    document.getElementById('likeCount')      §JS
      .textContent = likes;                   §JS
    document.getElementById('title')          §JS
      .textContent = `${likes} likes`;        §JS
  });                                         §JS
```

Plain JavaScript — you manage every DOM update manually. You find and update every element that depends on the same value.

JavaScript
React
HTML/JSX

Virtual DOM and reconciliation

```
function LikeCounter() {                          §JS
  const [likes, setLikes] = useState(0);          §REACT
  return (                                        §REACT
    <div><h1>                                     §HTML
      {                                           §REACT
        likes                                     §JS
      }                                           §REACT
      likes</h1>                                  §HTML
      <p>                                         §HTML
      {                                           §REACT
        likes                                     §JS
      }                                           §REACT
      </p><button                                 §HTML
        onClick= {                                §REACT
          () => setLikes(likes + 1)               §JS
        }                                         §REACT
      >Like</button></div>                        §HTML
  );                                              §REACT
}                                                 §JS
```

In React state changes trigger updates. Your code produces a representation of the UI, React creates a virtual copy and calculates minimum changes to update DOM.

JavaScript
React
HTML/JSX

React ecosystem

React Router for multi-page SPAs
- solves navigation between pages in React apps
- plain JavaScript uses browser navigation directly

Component libraries (e.g. React Bootstrap, Material UI, Headless UI) for reusable UI
- provides ready-made components
- plain HTML/CSS uses class names for the same styles

Development tools (e.g. Vite) for JSX transpilation
- transpiles JSX to JavaScript
- plain JavaScript needs no transpilation

JavaScript in React code
- Which parts of React code are JavaScript and which are React-specific?
- Which JavaScript techniques are essential for writing React components?
What is purely React?
- What does React add that JavaScript alone does not have?
What is purely JavaScript?
- Which parts of React code work exactly the same outside React?
- What is the "border zone" — JavaScript that behaves differently inside React?
From JavaScript to React — same app, different approach
- How does the same feature look in plain JavaScript vs. React?
- What changed, and what stayed the same?

JS that works the same outside React

```
cats.filter(cat => cat.age > 5);          §JS

cart.reduce((sum, item) =>                §JS
  sum + item.price, 0);                   §JS


const randomColor =                       §JS
  colors[Math.floor(                      §JS
    Math.random() * colors.length         §JS
  )];                                     §JS


console.log("Current state:", posts); §JS
```

These work the same inside and outside React — return new values without modifying original:

- Non-mutating array methods: map(), .filter(), .reduce(), .find()
- Spread operator, destructuring
- String methods
- Math operations, timers
- fetch() and async/await
- Debugging with console.log

Note: Methods that modify the original (.push(), .sort(), .reverse()) work in plain JavaScript. But are unsafe for React state, use spread or new arrays instead.

JS that behaves differently in React

Some JavaScript features work the same in React,
but their context or wrapping changes:

- .map() + key — .map() is JS, key is React
- event.preventDefault() — same function, different context
- event naming: onClick (camelCase on React) vs. onclick (lowercase on HTML)
- fetch() — same function, but needs useEffect wrapper

.map() — key for component rendering

Plain JS — .map() works the same way:

```
const items = data.map(item => item * 2);          §JS
// [2, 4, 6, 8]                                    §JS
```

In React — .map() renders components, needs key:

```
items.map((item, i) =>                             §JS
  <div                                             §HTML
    key= {                                         §REACT
      i                                            §JS
    }                                              §REACT
  >                                                §HTML
  {                                                §REACT
    item                                           §JS
  }                                                §REACT
  </div>                                           §HTML
)                                                  §JS
```

The key helps React track which element changed when the list updates. Without a key, React may re-render or misplace items when the list changes.

JavaScript
React
HTML/JSX

Event handling — different syntax

Plain JS — attach handler with addEventListener

```
form.addEventListener('submit', (e) => {          §JS
  e.preventDefault();                             §JS
});                                               §JS
```

React — event handler as a JSX prop:

```
<form                                             §HTML
  onSubmit=                                       §REACT
  {                                               §REACT
    (e) => {                                      §JS
      e.preventDefault();                         §JS
    }                                             §JS
  }                                               §REACT
>                                                 §HTML
```

Note: React uses camelCase (onClick, onSubmit) instead of HTML's lowercase (onclick, onsubmit).

JavaScript
React
HTML/JSX

fetch() — needs useEffect wrapper

Plain JS — call fetch when the script runs:

```
const res = await fetch(url);                    §JS
const data = await res.json();                   §JS
```

React — call fetch inside useEffect:

```
useEffect(() => {                                §REACT
  const getData = async () => {                  §JS
    const res = await fetch(url);                §JS
    const data = await res.json();               §JS
    setProducts(data);                           §REACT
  };                                             §JS
  getData();                                     §JS
}, []);                                          §REACT
```

React runs the component function on every render. Without useEffect, fetch would run repeatedly, causing an infinite loop.

JavaScript
React
HTML/JSX

JavaScript in React code
- Which parts of React code are JavaScript and which are React-specific?
- Which JavaScript techniques are essential for writing React components?
What is purely React?
- What does React add that JavaScript alone does not have?
What is purely JavaScript?
- Which parts of React code work exactly the same outside React?
- What is the "border zone" — JavaScript that behaves differently inside React?

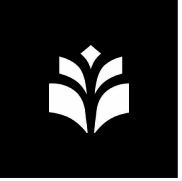
From JavaScript to React — same app, different approach
- How does the same feature look in plain JavaScript vs. React?
- What changed, and what stayed the same?

JavaScript
React
HTML/JSX

Adding a new post

```
<form id="postForm">                          §HTML
  <input id="title" />                        §HTML
  <button type="submit">Post</button>         §HTML
</form>                                       §HTML
<div id="posts"></div>                        §HTML
<script>                                      §HTML
const form = document                         §JS
  .getElementById('postForm');                §JS
form.addEventListener('submit', (e) => {      §JS
  e.preventDefault();                         §JS
  const title = document                      §JS
    .getElementById('title').value;           §JS
  const div = document                        §JS
    .createElement('div');                    §JS
  div.textContent = title;                    §JS
  document.getElementById('posts')            §JS
    .appendChild(div);                        §JS
});                                           §JS
</script>                                     §HTML
```

```
function App() {                              §JS
  const [posts, setPosts] = useState([]);     §REACT
  const [title, setTitle] = useState('');     §REACT
  const handleSubmit = (e) => {               §JS
    e.preventDefault();                       §JS
    setPosts([...posts, title]);              §REACT
    setTitle('');                             §REACT
  };                                          §JS
  return (                                    §REACT
    <div>                                     §HTML
      <form onSubmit={handleSubmit}>          §HTML
        <input                                §HTML
          value={title}                       §REACT
          onChange={(e) =>                    §REACT
            setTitle(e.target.value)}         §JS
        />                                    §HTML
        <button type="submit">Post</button>   §HTML
      </form>                                 §HTML
      {posts.map((p, i) =>                    §REACT
        <div key={i}>{p}</div>                §REACT
      )}                                      §REACT
    </div>                                    §HTML
  );                                          §REACT
}                                             §JS
```



What changed, what stayed the same?

	JavaScript	React
How you update UI	You make DOM changes directly with code	Your code produces a representation of the UI, React uses it to update the DOM
DOM manipulation	You do it manually (createElement, appendChild)	React does it for you (virtual DOM diffing)
State	You track it yourself (variables)	useState tracks it and triggers re-render
Event handling	AddEventListener separately	onClick, onSubmit as JSX props
What stayed the same?	.map(), .filter(), spread, ternary, arrow functions, template literals, fetch, async/await	

Back to the beginning

```
function PostCard({ title, caption, likes }) {
 const [liked, setLiked] = useState(false);
 return (
  <div className="post-card"style={{ padding: 10, margin: 5 }} >
    <h3> { title } </h3>
     <p> { caption} </p>
     <p> { likes } likes </p>
     <button onClick={ () => setLiked(!liked) } >
        { liked ? "Unlike" : "Like" }
     </button>
  </div>
  );
}
export default PostCard;
```

Example: PostCard

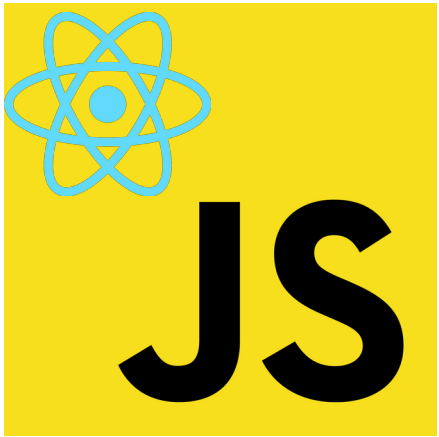
```
function PostCard({ title, caption, likes }) {
 const [liked, setLiked] = useState(false);
 return (
  <div className="post-card"style={ { padding: 10, margin: 5 } }
>
    <h3> { title } </h3>
     <p> { caption} </p>
     <p> { likes } likes </p>
     <button onClick={ () => setLiked(!liked) } >
        { liked ? "Unlike" : "Like" }
     </button>
  </div>
  );
}
export default PostCard;
```

JavaScript
React
HTML/JSX

React code is JavaScript

Previous PostCard component with JavaScript (function, destructuring, ternary operator, arrow function, object literal for style), React-specific (useState, JSX syntax with {}, JSX interpolation for attribute values) and HTML-like tags in JSX: , <h3>, <p>, <button> → Babel transpiles all the JSX into React.createElement() calls before the code runs in the browser. The final code that runs is pure JavaScript that calls React's functions.

The tag never reaches the browser as HTML. It becomes:
React.createElement('div', { className: 'post-card', style: { padding: 10, margin: 5 } }, ...)



React is a JavaScript library — not a new language. Learning React means learning:
- new JavaScript patterns (hooks like useState, useEffect)
- a syntax extension (JSX) that transpiles back to JavaScript
- a component-based way of thinking

Everything you write is still JS.
React just adds tools that make it easier to build interactive user interfaces.